



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University
(Estd. under Karnataka Act. No. 40 of 2013)

Project Report on

**Project Title: Legal Document Category
Classification**

Subject Title: Computer Organization

Subject code: CS2101T

Submitted By:

Student Name: Abhishek Krashna Pandre

University Enrolment Number: RTU24101IT014

Program Name: B. TECH IT AIML

Semester: 5th Sem

Session: 2026 – 2027

Submitted To:

Faculty Name: Mr. Abhishek S

BE, MTech

Department of Computer Science & Engineering

Rai Technology University.



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University

(Estd. under Karnataka Act. No. 40 of 2013)

College of Engineering & Application

CERTIFICATE

This is to certify that **Mr.Abhishek Krashna Pandre**, bearing UEN Number **RTU24101IT014**, has successfully completed the project entitled “**Legal Document Category Classification**” under the guidance and supervision of **Mr.ABHISHEK S**, Department of Computer Science and Engineering, during the academic year 2026–2027. The project work has been carried out by the student as part of the academic requirements of the B. Tech program, and the work presented in this report is completed satisfactorily to the best of our knowledge.

Head of Department

Name: _____

Sign: _____

Faculty Name

Name: _____

Sign: _____

ABSTRACT

Legal Document Category Classification using TF-IDF, Linear SVM, Logistic Regression, and Naive Bayes

The increasing availability of digital legal documents has created a need for efficient and automated methods to organize and classify legal information. Legal documents such as contracts, court judgments, agreements, notices, and case-related documents contain large amounts of unstructured textual data, making manual classification time-consuming and error-prone. This project presents a **Legal Document Category Classification** system using **Natural Language Processing (NLP)** and machine learning techniques to automatically classify legal documents into predefined categories.

The proposed system uses **Term Frequency–Inverse Document Frequency (TF-IDF)** to convert textual legal documents into numerical feature vectors. TF-IDF identifies important words in a document while reducing the importance of commonly occurring words. After feature extraction, three supervised machine learning algorithms—**Linear Support Vector Machine (SVM)**, **Logistic Regression**, and **Naive Bayes**—are applied for document classification. The models are trained using labelled legal document datasets and evaluated using performance measures such as **accuracy, precision, recall, and F1-score**.

The system follows several stages, including data collection, text preprocessing, tokenization, removal of unnecessary words, TF-IDF feature extraction, model training, prediction, and performance evaluation. The three algorithms provide different approaches to text classification: Linear SVM identifies an effective decision boundary between document categories, Logistic Regression estimates class probabilities, while Naive Bayes uses probabilistic relationships between words and categories. Comparing these models helps determine their effectiveness for automated legal document classification.

The developed system can reduce manual effort, improve document organization, and support faster retrieval of relevant legal information. It can be useful for **law firms, legal departments, courts, researchers, and academic institutions** where large collections of legal documents need to be categorized efficiently. Overall, the project demonstrates how NLP and machine learning can be combined to create an automated and practical solution for **legal text classification**.

ACKNOWLEDGEMENT

First and the foremost, I am highly grateful to **Dr. Arvind Dhar**, In-Charge Vice Chancellor, RAI TECHNOLOGY UNIVERSITY for providing an opportunity to fulfill the partial requirement towards obtaining our master degree by accomplishing the assignment.

I am highly grateful to **Dr. Nityananda B Patil**, Registrar, RAI TECHNOLOGY UNIVERSITY for providing an opportunity to fulfill the partial requirement towards obtaining our master degree by accomplishing the assignment.

I am highly grateful to **Mr. Kumar Sathyam Tanti**, Assistant professor and HOD for the kind support, guidance and encouragement throughout the course of this assignment.

Last but not least, I express our gratitude to our friends and family for their understanding and unwavering support during our academic journey.

ABHISHEK KRASHNA PANDRE

RTU24101IT014

CONTENTS

SL No.	TOPIC	Page No.
1.	Abstract	i
2.	Acknowledgement	ii
3.	Contents	iii
	Assignment-1	
4.	Introduction	1
5.	Problem Statement	2
6.	Literature Survey	3-5
	Assignment-2	
7.	Proposed Solution Methodology	6-8
8.	Data Collection and Processing	9-10
9.	Solution Development	11-15
	Assignment-3	
10.	Result Presentation	16-20
11.	Discussion on Findings	21-22
12.	Conclusion	23
13.	References	24

CHAPTER 1

ASSIGNMENT 1

INTRODUCTION

Legal documents contain specialized terminology, formal expressions, references to rights and obligations, procedural descriptions, and domain-specific concepts. As the number of digital legal records grows, manually reading and assigning every document to a legal category becomes time-consuming and difficult to scale. Automated text classification provides a practical way to organize such documents using natural language processing (NLP) and machine learning.

This project, titled “Legal Document Category Classification using TF-IDF and Machine Learning”, develops a text classification pipeline using Term Frequency–Inverse Document Frequency (TF-IDF) features with three supervised learning algorithms: Linear Support Vector Machine (Linear SVM), Logistic Regression, and Multinomial Naive Bayes. The uploaded 240-record dataset contains 15 legal categories and is used as the primary experimental dataset.

The project follows the same chapter-and-assignment style as the supplied “Abhishek Final” report, while replacing its social-media scalability topic with the legal-document classification problem. The supplied Python notebook and Streamlit program provide the implementation basis, including preprocessing, train-test splitting, TF-IDF vectorization, model training, accuracy evaluation, classification reports, confusion matrices, and new-document prediction.

- Automate legal document category assignment from textual content.
- Convert legal text into numerical TF-IDF features suitable for machine learning.
- Compare Linear SVM, Logistic Regression, and Multinomial Naive Bayes.
- Provide an interactive Streamlit interface for dataset analysis and prediction.
- Measure model performance using accuracy, precision, recall, F1-score, and confusion matrices.

CHAPTER 2

PROBLEM STATEMENT

Legal organizations, students, researchers, and information-management systems may handle large collections of contracts, criminal matters, property records, tax documents, labour documents, corporate documents, and other legal materials. When category assignment is performed manually, the process can require considerable human effort and may become inconsistent as the document collection grows.

The central problem addressed in this project is the automatic classification of a legal document into its appropriate category based only on its textual content. The system must learn patterns from labelled examples and use those patterns to predict the category of previously unseen documents.

- Legal text is highly domain-specific and contains words whose meaning depends on context.
- Different categories can share common vocabulary, creating classification ambiguity.
- The dataset contains multiple categories with unequal numbers of records, so stratified splitting is used to preserve category representation.
- Raw text cannot be supplied directly to the selected machine-learning algorithms; it must first be converted into numerical features.
- An effective solution should not only train a model but also provide measurable evaluation and a practical prediction interface.

The uploaded notebook implements a 20% test split with `random_state=42` and stratification. The Streamlit implementation additionally allows the user to upload a CSV file and select the text and category columns dynamically. □ This report uses the uploaded 240-record dataset for the reported numerical results. □

CHAPTER 3

OBJECTIVES, SCOPE AND LITERATURE SURVEY

Objectives

- Collect and inspect a labelled legal-document dataset.
- Clean missing and empty text records.
- Separate the document text from the target legal category.
- Create TF-IDF unigram and bigram features.
- Train Linear SVM, Logistic Regression, and Multinomial Naive Bayes classifiers.
- Compare model accuracy on a held-out test set.
- Analyze classification reports and the Linear SVM confusion matrix.
- Deploy the workflow through Streamlit for interactive classification.

Scope

The project focuses on text-based single-label classification. It does not attempt to provide legal advice, determine the legal validity of a document, or replace professional legal review. Its role is information organization: given a document and a set of learned categories, the system predicts the most likely category represented in the training data.

Literature Survey

The supplied project materials emphasize the use of TF-IDF and conventional supervised learning for document classification. In the implementation, TF-IDF is combined with Linear SVM, Logistic Regression, and Multinomial Naive Bayes. This provides a transparent baseline for comparing linear discriminative and probabilistic approaches before considering more computationally intensive language models.

CHAPTER 3

LITERATURE SURVEY – TECHNICAL FOUNDATION

TF-IDF is a standard representation for document classification. It increases the weight of terms that are important within a document while reducing the relative importance of terms that occur frequently across the complete corpus. In the uploaded notebook, the vectorizer uses lowercase=True, English stop-word removal, a maximum of 10,000 features, unigrams and bigrams (ngram_range=(1,2)), and min_df=1.

Linear SVM is a supervised classifier that searches for a separating decision boundary in the feature space. It is particularly suitable for sparse, high-dimensional text representations because TF-IDF creates a large number of numerical features.

Logistic Regression models class probabilities through a linear decision function. It provides a useful comparison because it is simple, widely used for text classification, and can work effectively with sparse TF-IDF features.

Multinomial Naive Bayes is a probabilistic text classifier based on conditional independence assumptions. It is computationally lightweight and often provides a strong baseline for word-frequency-based classification.

Technique	Role in Project	Main Characteristics
TF-IDF	Feature extraction	Sparse numerical representation of legal text
Linear SVM	Classifier	Linear decision boundary for high-dimensional text
Logistic Regression	Classifier	Linear probabilistic classification
Multinomial Naive Bayes	Classifier	Probability-based baseline for text features

CHAPTER 3

RESEARCH GAP AND PROPOSED DIRECTION

A practical legal-document classification system must connect text preprocessing, feature extraction, classification, evaluation, and user interaction into one repeatable workflow. A model accuracy value alone is not sufficient: the system should expose category distribution, training/test sizes, model comparisons, classification reports, confusion matrices, and predictions for new text.

The supplied Streamlit program addresses this integration gap by combining dataset upload, column selection, cleaning, category distribution, TF-IDF training, three classifiers, accuracy metrics, classification reports, confusion matrices, and an input box for new-document prediction. This creates a complete demonstration pipeline rather than an isolated training script.

- Data-level gap: legal categories can be represented by different numbers of examples.
- Feature-level gap: legal text must be transformed into a sparse numerical representation before training.
- Model-level gap: different algorithms can behave differently on the same TF-IDF representation.
- Evaluation gap: accuracy should be supported by per-class precision, recall, F1-score, and confusion analysis.
- Deployment gap: a trained model becomes more useful when a user can upload data and classify new text through a simple interface.

The proposed direction is therefore an end-to-end NLP classification application that keeps the original project terminology and implementation approach while presenting the results in the structured chapter format of the supplied report.

CHAPTER 4

SYSTEM ARCHITECTURE

The architecture follows a sequential NLP workflow. Each stage produces an output consumed by the next stage.

Stage	Input	Processing	Output
1. Data Loading	CSV dataset	Read with pandas	DataFrame
2. Cleaning	Raw rows	Drop null/blank text	Clean labelled data
3. Split	Text + labels	80/20 stratified split	Train/Test sets
4. Feature Extraction	Train/Test text	TF-IDF unigram + bigram	Sparse matrices
5. Training	Training matrix + labels	Train 3 classifiers	Fitted models
6. Evaluation	Test matrix + labels	Predict and score	Metrics
7. Application	User text / CSV	Transform and predict	Category prediction

The Streamlit application follows the same architecture and adds user-facing controls. The uploaded Python file defines a wide page layout, a sidebar file uploader, dataset overview, selectable text/category columns, model performance metrics, a model selector for classification reports and confusion matrices, and a text area for predicting a new legal document.

The architecture is intentionally modular. TF-IDF is fitted only on the training text and then applied to the test text and new documents. This prevents the test vocabulary from being used to fit the feature extractor.

CHAPTER 5

DATA COLLECTION AND PROCESSING

The primary experimental file is “legal_documents_240_mixed(1).csv”. It contains 240 records and three columns: document_id, text, and category. For model training, the notebook selects only text and category. The document_id is retained in the raw dataset for identification but is not used as a predictive feature.

Dataset Property	Observed Value
Total records	240
Columns	3 (document id, text, category)
Model input columns	text, category
Number of categories	15
Training records	192
Testing records	48
TF-IDF feature matrix	192×1549 (training)
Test TF-IDF matrix	48×1549

Category distribution

Legal Category	Documents
Arbitration Law	10
Banking Law	10
Civil	20
Constitutional Law	10
Consumer Law	10
Contract	20
Corporate Law	20
Criminal	20
Environmental Law	10
Family Law	20
Insurance Law	10
Intellectual Property	20
Labour Law	20
Property Law	20
Tax Law	20

The dataset is mixed across legal domains including Criminal, Contract, Civil, Corporate Law, Family Law, Intellectual Property, Labour Law, Property Law, Tax Law, Constitutional Law, Environmental Law, Arbitration Law, Consumer Law, Banking Law, and Insurance Law.

CHAPTER 5

DATA PREPROCESSING

Preprocessing converts the raw dataset into a reliable input for the classification stage. The uploaded notebook performs the following steps:

- Select the text and category columns.
- Remove rows containing missing values using `dropna()`.
- Convert the text column to string format.
- Remove records whose text becomes empty after stripping whitespace.
- Separate the cleaned text as `X` and category labels as `y`.
- Perform a stratified train-test split so the category proportions are represented in both subsets.

The Streamlit implementation performs equivalent cleaning dynamically after the user selects the text and category columns. It also checks that the selected target column contains at least two categories and reports an error if no usable text remains.

Preprocessing is important because a classifier can only learn from meaningful input. Blank records do not provide useful lexical evidence, while inconsistent data types can create problems during vectorization. Stratification is especially useful for this dataset because some categories contain 10 documents while others contain 20.

CHAPTER 5

TF-IDF FEATURE EXTRACTION

TF-IDF represents each document as a vector of term weights. The term-frequency component reflects how strongly a term appears in a document, while the inverse-document-frequency component reduces the influence of terms that appear in many documents.

The uploaded notebook configures the vectorizer as follows: lowercase=True, stop_words='english', max_features=10000, ngram_range=(1,2), and min_df=1. Unigrams capture individual terms while bigrams capture two-word expressions that can carry useful legal meaning.

Parameter	Value	Purpose
lowercase	True	Normalize capitalization
stop_words	english	Remove common English stop words
max_features	10,000	Limit vocabulary size
ngram_range	(1,2)	Use unigrams and bigrams
min_df	1	Keep terms occurring in at least one document

For the uploaded 240-record dataset, the resulting training matrix contains 1,549 features. The actual number is below the 10,000 maximum because the corpus contains a smaller number of unique retained n-grams after preprocessing.

The same fitted TF-IDF vectorizer is used for all three classifiers. This makes the model comparison more controlled because the algorithms receive the same feature representation.

CHAPTER 6

MODEL DEVELOPMENT – LINEAR SVM

The first classifier is Linear Support Vector Machine. The uploaded notebook creates `LinearSVC()`, fits it using the training TF-IDF matrix and training labels, and predicts the categories of the test documents.

Using the uploaded 240-record dataset and the notebook settings, Linear SVM achieved an accuracy of 95.83% on the 48-document test set. This corresponds to 46 correct predictions and 2 incorrect predictions.

Metric	Linear SVM Result
Test documents	48
Correct predictions	46
Incorrect predictions	2
Accuracy	95.83%
Macro F1	0.93
Weighted F1	0.95

The two observed Linear SVM errors in this run were one Arbitration Law document predicted as Environmental Law and one Banking Law document predicted as Property Law. These errors illustrate that semantically related legal language can create overlap between categories.

CHAPTER 6

MODEL DEVELOPMENT – LOGISTIC REGRESSION AND NAIVE BAYES

Logistic Regression is configured with `max_iter=5000` in the notebook. On the same 48-document test set, it achieved 81.25% accuracy, corresponding to 39 correct predictions.

Multinomial Naive Bayes was trained using the same TF-IDF matrix. It achieved 77.08% accuracy, corresponding to 37 correct predictions.

Model	Accuracy	Correct / 48
Linear SVM	95.83%	46/48
Logistic Regression	81.25%	39/48
Naive Bayes	77.08%	37/48

These values are the results of one specified train-test split (`random_state=42`) on the uploaded dataset; they should not be interpreted as universal performance for legal-document classification. Different datasets, category definitions, preprocessing choices, or splits can produce different results.

The comparison is useful because all three algorithms use the same TF-IDF representation and test set. Therefore, the observed difference comes from the classification algorithms under this experimental configuration rather than from different input features.

CHAPTER 6

STREAMLIT APPLICATION DEVELOPMENT

The uploaded Streamlit program converts the notebook workflow into an interactive application. The page title is “Legal Document Classification”, with the subtitle “TF-IDF + Linear SVM + Logistic Regression + Naive Bayes”. A sidebar allows the user to upload a CSV file.

- Dataset Overview – displays the dataset shape and first 10 rows.
- Dataset Columns – lets the user select the text and category columns.
- Legal Document Categories – shows category counts and a distribution chart.
- Training Data Summary – reports training and testing document counts.
- Model Performance – displays the accuracy of the three classifiers.
- Accuracy Comparison – presents a bar chart.
- Classification Reports – allows the user to select a model and inspect precision, recall and F1-score.
- Confusion Matrix – displays actual versus predicted categories.
- Predict Legal Document Category – accepts new text and reports predictions from all three models.

The application also includes validation for empty text, insufficient target categories, and stratified splitting failures. This improves usability because the user receives a meaningful message instead of an unhandled exception.

CHAPTER 6

CODE IMPLEMENTATION

The implementation is based directly on the uploaded Python and notebook files. The core imports include pandas, matplotlib, seaborn, train_test_split, TfidfVectorizer, LinearSVC, LogisticRegression, MultinomialNB, accuracy_score, classification_report, and confusion_matrix.

```
df = pd.read_csv("legal_documents_240_mixed.csv")
X = df["text"]
y = df["category"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
tfidf = TfidfVectorizer(
    lowercase=True, stop_words="english",
    max_features=10000, ngram_range=(1, 2), min_df=1
)
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

Note: the uploaded notebook references the filename legal_documents_240_mixed.csv, while the uploaded file available for this report is named legal_documents_240_mixed.csv. The data content used for the reported results is the uploaded 240-record file.

CHAPTER 6

MODEL TRAINING AND PREDICTION CODE

```
svm_model = LinearSVC()
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)
svm_accuracy = accuracy_score(y_test, svm_predictions)
```

```
logistic_model = LogisticRegression(max_iter=5000)
logistic_model.fit(X_train_tfidf, y_train)
logistic_predictions = logistic_model.predict(X_test_tfidf)
```

```
nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)
nb_predictions = nb_model.predict(X_test_tfidf)
```

For a new document, the same fitted TF-IDF vectorizer transforms the text before each classifier makes a prediction. The example in the notebook describes an agreement containing obligations and terms; in the current run all three models classify that sample as Contract.

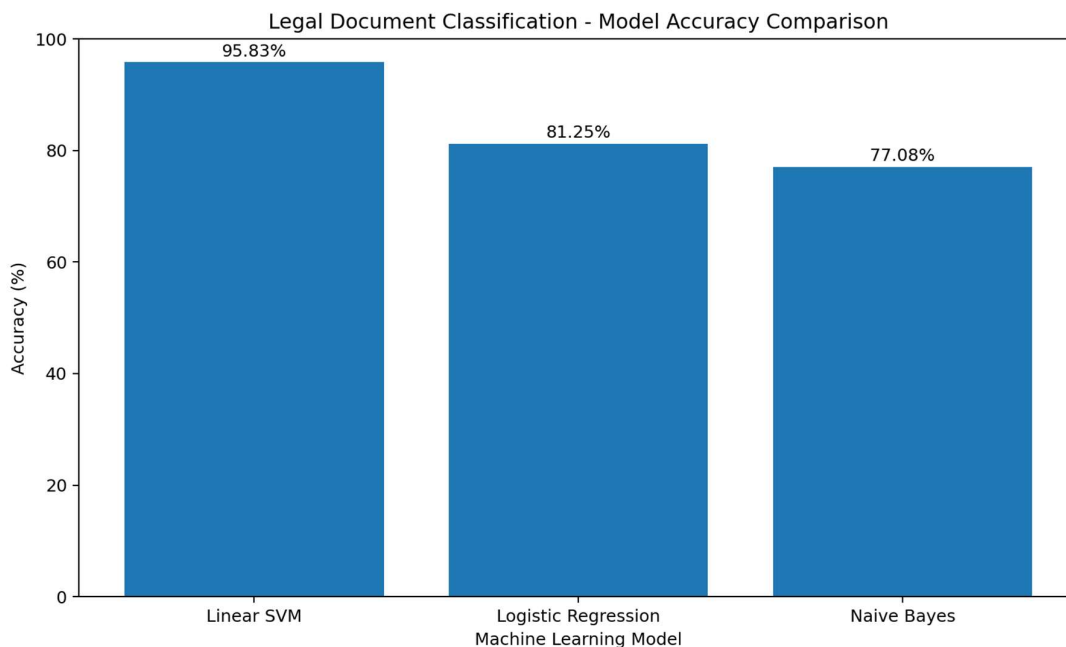
This demonstrates the complete inference path: raw text → TF-IDF vector → trained model → predicted legal category.

CHAPTER 7

ASSIGNMENT 3

RESULT PRESENTATION

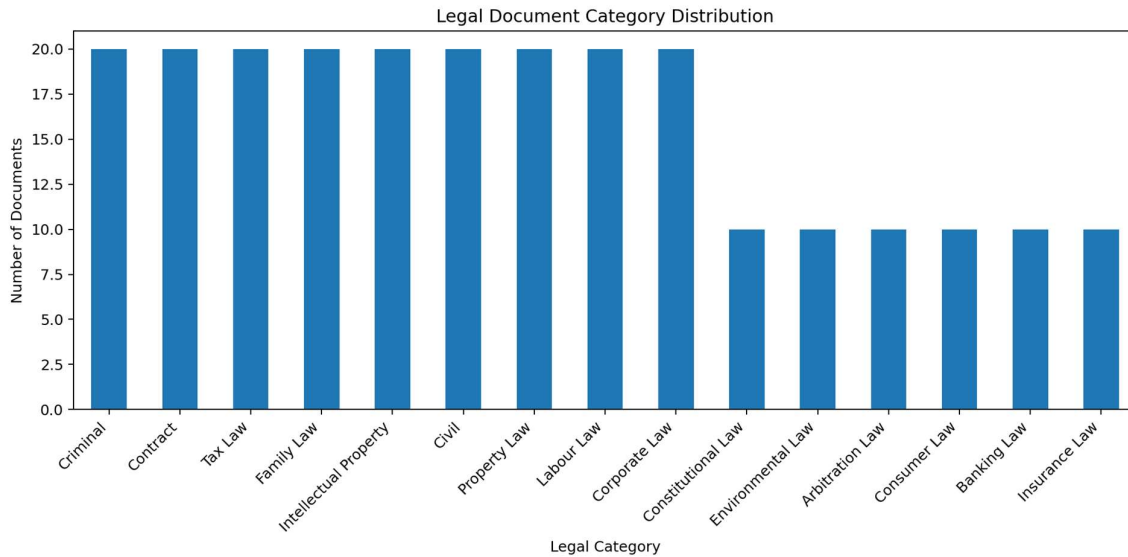
The experimental results are presented using dataset statistics, model accuracy, classification reports, and confusion-matrix analysis. The test set contains 48 records. Because the split is stratified, each category is represented according to its availability in the source dataset.



The observed accuracies are 95.83% for Linear SVM, 81.25% for Logistic Regression, and 77.08% for Multinomial Naive Bayes on this specific test split. These values are derived by rerunning the uploaded notebook methodology against the uploaded 240-record dataset.

CHAPTER 7

DATASET DISTRIBUTION



The uploaded dataset contains 15 categories. The largest groups contain 20 documents, while several categories contain 10 documents. The category distribution is therefore not perfectly uniform, which is one reason stratified splitting is appropriate.

Category	Count
Arbitration Law	10
Banking Law	10
Civil	20
Constitutional Law	10
Consumer Law	10
Contract	20
Corporate Law	20
Criminal	20
Environmental Law	10
Family Law	20
Insurance Law	10
Intellectual Property	20
Labour Law	20
Property Law	20
Tax Law	20

CHAPTER 7

MODEL COMPARISON

Model	Accuracy (%)	Correct	Incorrect
Linear SVM	95.83	46	2
Logistic Regression	81.25	39	9
Naive Bayes	77.08	37	11

The comparison shows different behaviour under the same feature representation. Linear SVM produced the highest measured accuracy in this experiment, while Logistic Regression and Naive Bayes produced lower measured accuracy on the same test set. This is an empirical observation for the uploaded dataset and configuration, not a general claim that one algorithm will always perform better on every legal corpus.

The notebook's model-comparison table and accuracy bar chart are reproduced conceptually in this report. The Streamlit program also calculates these values dynamically whenever the dataset is processed.

CHAPTER 7

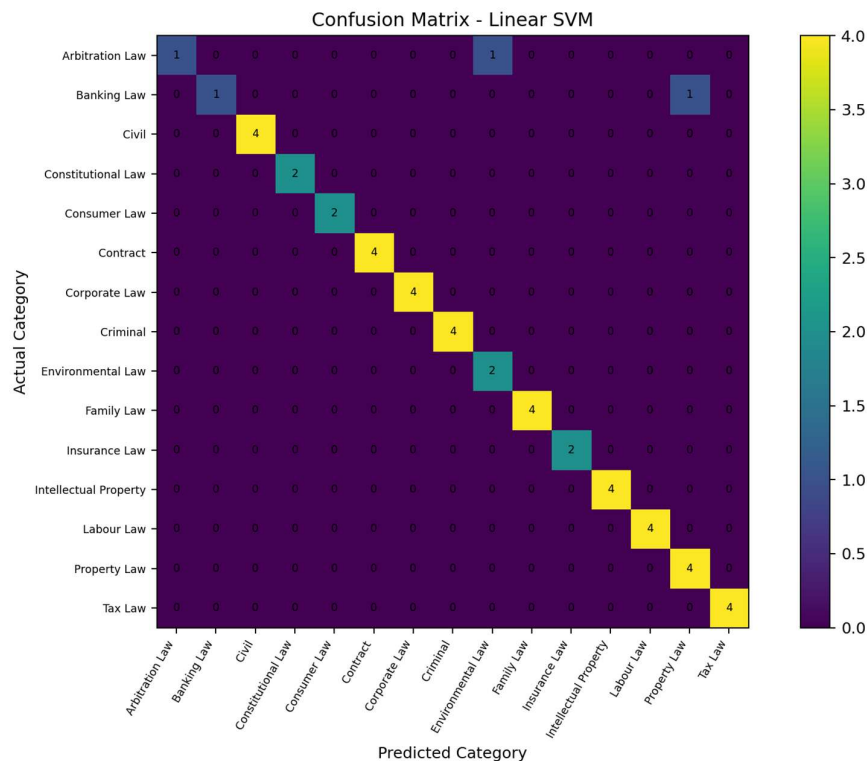
CLASSIFICATION REPORT ANALYSIS

Category	Precision	Recall	F1	Support
Arbitration Law	1.00	0.50	0.67	2
Banking Law	1.00	0.50	0.67	2
Civil	1.00	1.00	1.00	4
Constitutional Law	1.00	1.00	1.00	2
Consumer Law	1.00	1.00	1.00	2
Contract	1.00	1.00	1.00	4
Corporate Law	1.00	1.00	1.00	4
Criminal	1.00	1.00	1.00	4
Environmental Law	0.67	1.00	0.80	2
Family Law	1.00	1.00	1.00	4
Insurance Law	1.00	1.00	1.00	2
Intellectual Property	1.00	1.00	1.00	4
Labour Law	1.00	1.00	1.00	4
Property Law	0.80	1.00	0.89	4
Tax Law	1.00	1.00	1.00	4
Macro average	0.96	0.93	0.93	48
Weighted average	0.97	0.96	0.95	48

Precision describes how often predictions for a category are correct, recall describes how many documents belonging to a category are retrieved, and F1-score combines the two. The report is especially useful when categories have different numbers of examples.

CHAPTER 7

CONFUSION MATRIX AND ERROR ANALYSIS



The Linear SVM confusion matrix contains 46 correct predictions on the diagonal and two off-diagonal errors in this run. One Arbitration Law test document was predicted as Environmental Law, and one Banking Law test document was predicted as Property Law.

The confusion matrix is valuable because it reveals which categories are being confused rather than only reporting one overall accuracy number. In a larger or more heterogeneous legal corpus, similar analysis could guide collection of additional training examples for frequently confused categories.

CHAPTER 8

DISCUSSION ON FINDINGS

The experimental workflow demonstrates that a conventional TF-IDF-based NLP pipeline can classify the supplied legal documents with measurable performance. The Linear SVM result was 95.83% on the selected test split, while Logistic Regression and Naive Bayes produced 81.25% and 77.08%, respectively.

- The dataset is relatively small at 240 records, so individual examples can have a noticeable effect on measured accuracy.
- The 20% test split contains only 48 records; therefore a small number of errors changes the percentage substantially.
- Several categories contain only 10 documents, limiting the number of test examples available for those categories.
- TF-IDF captures lexical patterns but does not explicitly model deep semantic relationships or long-range context.
- The high Linear SVM result should be interpreted as an experiment-specific measurement rather than a universal benchmark.

The application design is also significant. By combining data upload, cleaning, feature extraction, model comparison, reporting, and prediction, the project demonstrates how an ML experiment can be converted into a user-facing prototype.

CHAPTER 8

ADVANTAGES, LIMITATIONS AND FUTURE WORK

Advantages

- Simple and reproducible preprocessing pipeline.
- Three classifiers provide a direct comparative experiment.
- TF-IDF supports sparse, high-dimensional legal text.
- Streamlit provides an accessible graphical interface without requiring a separate frontend.
- Classification reports and confusion matrices support deeper evaluation than accuracy alone.

Limitations

- The dataset is small compared with real-world legal repositories.
- The categories and examples are dependent on the supplied dataset.
- The system does not use document metadata, citations, legal statutes, or structured case information.
- TF-IDF does not provide the contextual understanding available from transformer-based language models.
- The current Streamlit application retrains models when the application processes the dataset rather than loading pre-trained model artifacts.

Future Work

- Expand the dataset with more legally reviewed examples.
- Use cross-validation and repeated experiments for more stable estimates.
- Compare TF-IDF with word embeddings and transformer-based representations.
- Add model persistence so trained models can be reused without retraining.
- Add explainability features such as influential terms or confidence-oriented outputs where supported.
- Add document upload for TXT/PDF extraction while maintaining appropriate privacy controls.

CHAPTER 9

CONCLUSION

This project presents an end-to-end Legal Document Category Classification system using TF-IDF and three machine-learning algorithms: Linear SVM, Logistic Regression, and Multinomial Naive Bayes. The work follows the structure of the supplied “Abhishek Final” report while adapting its content to the legal-document classification topic supported by the uploaded notebook, Python program, datasets, and presentation materials.

Using the uploaded 240-record dataset and the exact notebook configuration of an 80:20 stratified split, TF-IDF unigrams/bigrams, and the three specified classifiers, the measured test accuracies were 95.83% for Linear SVM, 81.25% for Logistic Regression, and 77.08% for Multinomial Naive Bayes.

The project also provides a practical Streamlit interface. It displays the dataset, category distribution, model metrics, classification report, confusion matrix, and predictions for new legal text. This connects the machine-learning experiment with a usable demonstration system.

Overall, the project establishes a clear baseline for automated legal document organization. Further improvement should focus on larger and more diverse labelled datasets, stronger validation procedures, contextual language representations, model persistence, explainability, and privacy-aware document handling.

CHAPTER 10

REFERENCES AND SOURCE FILES

Project Source Files Used

1. Abhishek Final.docx – supplied 24-page report used as the primary structural and formatting reference. Its original chapter sequence contains Introduction, Problem Statement, Literature Survey, Proposed Solution and Methodology, Solution Development, Result Presentation, Discussion on Findings, Conclusion, and References.
2. legal_document.ipynb – supplied Jupyter Notebook containing the TF-IDF, Linear SVM, Logistic Regression, Naive Bayes, accuracy comparison, classification report, confusion matrix, and new-document prediction workflow.
3. legal_document_classification.py – supplied Streamlit application implementing dataset upload, dynamic column selection, cleaning, model training, metrics, classification reports, confusion matrix, and new-document prediction.
4. legal_documents_240_mixed.csv – supplied 240-record dataset used for the numerical results in this report.
5. legal_documents.csv – supplied 60-record dataset containing four categories; retained as an additional dataset reference but not used for the reported 240-record experiment.
6. PPT format.pptx – supplied presentation format/reference containing the original topic's title, content, introduction, problem statement, literature survey, proposed solution, solution development, performance analysis, conclusion, and thank-you structure.

Methodology note: all numerical model results in this report were generated from the supplied 240-record dataset using the preprocessing and model settings represented in the uploaded notebook. The report does not treat the social-media-specific numerical tables in the original “Abhishek Final” document as legal-classification results.

END OF REPORT